Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 1.1 (Page 1 of 2) - **3.0 marks**

Submitted: 2025-11-20 20:04:26 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=589d1153-7a49-4d28-803f-60cbcdd52e8b

Student Answer:

Summary of Key Points for Converting Binary Sequences to Decimal:

Understand Bit Positions: Always number bits from right to left, starting at 0. Each bit represents a power of 2 corresponding to its position. Identify ‘1’ Bits:Only the bits set to 1 contribute to the total value. Bits set to 0 can be ignored in the sum. Compute Contributions:For each 1 bit, calculate (2^{\text{position}}) to find its decimal contribution.

Sum Carefully:Add all contributions together to get the final decimal value. Double-check each addition to avoid errors. Watch for Endianness: Be aware that how bytes are ordered in memory (big-endian vs little-endian) can change the interpretation.

Optional Shortcut:Converting binary $\rightarrow$ hexadecimal $\rightarrow$ decimal can simplify calculations for long sequences.

Pitfalls to Avoid:

Misnumbering bit positions (e.g., counting left-to-right instead of right-to-left). Forgetting which bits are actually 1. Ignoring endianness when interpreting multi-byte sequences.

By following these steps carefully, the 32-bit binary sequence 01000011010011110100010001000101 was correctly converted to the decimal value 1,129,268,293.

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 1.1 (Page 2 of 2) - **3.0 marks**

Submitted: 2025-11-20 20:04:26 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=589d1153-7a49-4d28-803f-60cbcdd52e8b

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 1.2 (Page 1 of 2) - **3.0 marks**

Submitted: 2025-11-20 20:09:21 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=e1ef033a-69f6-433a-85fd-b420a25e3465

Student Answer:

Split the 32-bit sequence into four bytes: 01000011, 01001111, 01000100, 01000101 Convert each byte to decimal: 01000011 → 67 01001111 → 79 01000100 → 68 01000101 → 69 Convert the decimal numbers to ASCII characters: 67 → 'C' 79 → 'O' 68 → 'D' 69 → 'E' Combine the characters to get the message: "CODE".

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 1.2 (Page 2 of 2) - **3.0 marks**

Submitted: 2025-11-20 20:09:21 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=e1ef033a-69f6-433a-85fd-b420a25e3465

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 1.3 (Page 1 of 2) - **4.0 marks**

Submitted: 2025-11-20 20:12:53 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=1cc38c1a-8e08-415f-8000-4ab68f17071f

Student Answer:

Big endian stores the most significant byte at the lowest memory address little endian stores the least significant byte at the lowest memory address

The 32 bit value 0x434F4445 is stored in memory starting at 0x1000

Big endian memory layout 0x1000 0x43 0x1001 0x4F 0x1002 0x44 0x1003 0x45

Little endian memory layout 0x1000 0x45 0x1001 0x44 0x1002 0x4F 0x1003 0x43

Endianness affects how multi byte values are stored and read in memory but the actual value is the same

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 1.3 (Page 2 of 2) - **4.0 marks**

Submitted: 2025-11-20 20:12:53 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=1cc38c1a-8e08-415f-8000-4ab68f17071f

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 2.1 (Page 1 of 2) - **5.0 marks**

Submitted: 2025-11-20 23:40:42 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=ff75ea67-442f-414d-9473-4f0b79eaca51

Student Answer:

The union must be large enough to hold its largest member: double = 8 bytes int status[3] = 12 bytes char id[7] = 7 bytes The largest member is 12 bytes, but the union must also satisfy the strictest alignment requirement (8 bytes, from double). Therefore, the size must be rounded up to the nearest multiple of 8: Size = 16 bytes Answer: 16 bytes b) Size of struct Packet Layout of the struct: offset 0–1: short type (2 bytes) offset 2–7: padding to align next field to 8 bytes offset 8–23: union Payload (16 bytes) offset 24: char priority (1 byte) offset 25–27: padding to align next int offset 28–31: int sequence (4 bytes) The final size is 32 bytes, and it is already a multiple of the largest alignment in the struct (8 bytes). Answer: 32 bytes

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 2.1 (Page 2 of 2) - **5.0 marks**

Submitted: 2025-11-20 23:40:42 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=ff75ea67-442f-414d-9473-4f0b79eaca51

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 2.2 (Page 1 of 2) - **5.0 marks**

Submitted: 2025-11-20 23:43:17 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=9bdcec1b-44f3-4dd6-9466-61bf489d0edf

Student Answer:

a) Offset of data.value The struct begins with short type at offset 0–1. The next field is union Payload, which must be aligned to 8 bytes (because its largest member is a double). Therefore, 6 bytes of padding are added at offsets 2–7. So, data.value begins at offset 8. b) Offset of sequence union Payload occupies 16 bytes (offsets 8–23). char priority is placed at offset 24. To align the following int (4-byte alignment), 3 bytes of padding are inserted (offsets 25–27). Therefore, sequence begins at offset 28.

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 2.2 (Page 2 of 2) - **5.0 marks**

Submitted: 2025-11-20 23:43:17 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=9bdcec1b-44f3-4dd6-9466-61bf489d0edf

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 2.3 (Page 1 of 2) - **5.0 marks**

Submitted: 2025-11-21 01:56:48 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=87c2e964-4396-46f9-9dd5-ef1150866bff

Student Answer:

Reordered struct (largest to smallest):

union Payload data int sequence short type char priority Memory layout: data → offset 0–7 sequence → offset 8–11 type→ offset 12–13 priority→ offset 14 padding→ offset 15 (to maintain 8-byte alignment)

Total size = 16 bytes (Reality/C Standard: union Payload is 16 bytes. (Because int status[3] is 12 bytes, and it must be padded to a multiple of the union's alignment, which is 8). This makes the total struct 24 bytes.)

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 2.3 (Page 2 of 2) - **5.0 marks**

Submitted: 2025-11-21 01:56:48 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=87c2e964-4396-46f9-9dd5-ef1150866bff

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 3.1 (Page 1 of 2) - **2.0 marks**

Submitted: 2025-11-20 20:32:57 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=6566c912-f13f-48ed-84d0-8d46ee155eec

Student Answer:

The expression inttostring(add(10, 5)) is typeable The inner add(10, 5) returns 15 (int) inttostring then converts 15 to "15" (string) So the final type is string and the expected output is "15"

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 3.1 (Page 2 of 2) - **2.0 marks**

Submitted: 2025-11-20 20:32:57 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=6566c912-f13f-48ed-84d0-8d46ee155eec

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 3.2 (Page 1 of 2) - **2.0 marks**

Submitted: 2025-11-20 20:35:22 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=9749a206-7f6d-4a8b-b35b-ae375a6b886f

Student Answer:

add("hello", "world") is not typeable because add requires int inputs but the expression gives strings the input types do not match the operator so it cannot be evaluated final answer: not typeable inputs are strings but add needs ints

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 3.2 (Page 2 of 2) - **2.0 marks**

Submitted: 2025-11-20 20:35:22 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=9749a206-7f6d-4a8b-b35b-ae375a6b886f

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 3.3 (Page 1 of 2) - **2.0 marks**

Submitted: 2025-11-20 20:36:56 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=b2a027b7-1e8c-4074-848b-cb91ed3cafb5

Student Answer:

greater_than(10, 5) is not typeable because the second argument should be a float but it is an int so the input types do not match the function definition

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 3.3 (Page 2 of 2) - **2.0 marks**

Submitted: 2025-11-20 20:36:56 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=b2a027b7-1e8c-4074-848b-cb91ed3cafb5

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 3.4 (Page 1 of 2) - **2.0 marks**

Submitted: 2025-11-21 01:59:54 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=852bd515-396f-4d22-b590-0536d0d5309a

Student Answer:

The expression is typeable. Reasoning: greater_than(20, 10.5) takes an int and float, which matches the definition. It returns a boolean (true). equals("hi", "hi") takes two strings, matching the definition. It returns a boolean (true). and(...) takes the two boolean results above, matching the definition and(boolean, boolean). Final Type: boolean Expected Output: true

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 3.4 (Page 2 of 2) - **2.0 marks**

Submitted: 2025-11-21 01:59:54 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=852bd515-396f-4d22-b590-0536d0d5309a

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 4.1 (Page 1 of 2) - **6.0 marks**

Submitted: 2025-11-21 17:41:00 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=9f286704-83aa-4f3a-940d-e4d9e924d3d0

Student Answer:

The term ((λx: int. (x+3) 5) 2) has type int Justification 1 Using T-Abs rule: λx: int. (x+3) *5 has type int → int because x is int and (x+3) *5 is int 2 Using T-App rule: applying this function to 2 is valid because 2 has type int which matches the function's input type 3 Therefore the whole expression ((λx: int. (x+3) *5) 2) has type int (because x is int, and by using the arithmetic rules (T-PLUS, T-MULT), the body (x+3)5 evaluates to type int) Term: ((λx: int. (x+3) * 5) 2) Final Type: int

Proof Tree:

(T-Var) (T-Int) ------- --------- Γ ■ x:int Γ ■ 3:int ---------- (T-Plus) Γ ■ (x+3):int Γ ■ 5:int ------------------------------ (T-Mult)

Context: Γ ■ (x+3) * 5 : int Γ = {x:int} ------------------------------------ (T-Abs) ■ (λx:int. (x+3) * 5) : int -> int ■ 2:int -- (T-App) ■ ((λx:int. (x+3) * 5) 2) : int

Term: ((λx: int. (x+3) * 5) 2) Final Type: int

Proof Derivation (Linear Format due to text editor limitations):

Context Setup: Let Γ={x:int} Analyze Body of Lambda:

Γ■x:int (Rule: T-VAR)

Γ■3:int (Rule: T-INT)

Γ■(x+3):int (Rule: T-PLUS on previous steps)

[Answer continues on next page...]

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 4.1 (Page 2 of 2) - **6.0 marks**

Submitted: 2025-11-21 17:41:00 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=9f286704-83aa-4f3a-940d-e4d9e924d3d0

Student Answer (continued):

Γ■5:int (Rule: T-INT)

Γ■((x+3)∗5):int (Rule: T-MULT on previous steps)

Type the Function (Abstraction):

Using Rule T-ABS on the body derived in Step 2:

■(λx:int.(x+3)∗5):int→int

Apply Argument (Application):

The argument is 2.

■2:int (Rule: T-INT)

The function takes int and the argument is int. Matches.

Using Rule T-APP:

Result: ■((λx:int.(x+3)∗5)2):int

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 4.2 (Page 1 of 2) - **6.0 marks**

Submitted: 2025-11-21 17:50:19 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=ba4e97a5-551c-4773-aac6-0ee8a24b8c29

Student Answer:

Term: ((λy:float.(y*(3+12))) 4.5) Final Type: float

Proof Tree:

(T-Int) (T-Int) ------- ------- (T-Var) Γ'■3:int Γ'■12:int ------- ---------- (T-Plus) Γ'■y:float Γ' ■ (3+12):int ---------- (T-Mult-FI)

Context: Γ' ■ (y * (3+12)) : float Γ' = {y:float} ---------- (T-Abs) ■ (λy:float. (y(3+12))) : float -> float ■ 4.5:float ---------- (T-App) ■ ((λy:float.(y(3+12))) 4.5) : float

Proof Derivation (Linear Format due to text editor limitations):

Context Setup: Let Γ' = {y : float} Analyze Inner Arithmetic:

Γ' ■ 3 : int (Rule: T-INT)

Γ' ■ 12 : int (Rule: T-INT)

Γ' ■ (3 + 12) : int (Rule: T-PLUS)

Analyze Multiplication (Mixed Types):

[Answer continues on next page...]

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 4.2 (Page 2 of 2) - **6.0 marks**

Submitted: 2025-11-21 17:50:19 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=ba4e97a5-551c-4773-aac6-0ee8a24b8c29

Student Answer (continued):

Γ' ■ y : float (Rule: T-VAR)

Γ' ■ (3 + 12) : int (From Step 2)

Rule Used: T-MULT-FI (Float * Int → Float)

Result: Γ' ■ (y * (3 + 12)) : float

Type the Function (Abstraction):

Using Rule T-ABS on the body derived in Step 3.

Function Type: ■ (λy:float. (y * (3 + 12))) : float → float

Apply Argument (Application):

The argument is 4.5.

■ 4.5 : float (Rule: T-FLOAT)

The function expects float input. The argument is float.

Using Rule T-App.

Final Result: ■ ((λy:float. (y * (3 + 12))) 4.5) : float

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 5 (Page 1 of 2) - **5.0 marks**

Submitted: 2025-11-20 21:03:37 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=1dd024cc-3ec3-4f85-ab58-3b45a7d0067e

Student Answer:

(Γ $\vdash$ e : bool Γ $\vdash$ s1 : stmt Γ $\vdash$ s2 : stmt)/(Γ $\vdash$ if e then s1 else s2 : stmt) * (IFTHENELSE.STMT). A perfect example would be:

if (x > 10) then (y := 5) else (z := 0)

The rule works because:

Condition (e): x > 10 is a boolean expression, satisfying the Γ■e:bool premise.

Then Branch (s 1 ■): y := 5 is an assignment statement, satisfying Γ■s 1 ■ :stmt.

Else Branch (s 2 ■): z := 0 is also an assignment statement, satisfying Γ■s 2 ■ :stmt.

Since all three premises hold, the rule IFTHENELSE.STMT allows us to conclude that the entire if-then-else structure is itself a well-formed statement (stmt).

Name: mma7020@psu.edu

Email: mma7020@psu.edu

Assignment: Assignment 6

Question 5 (Page 2 of 2) - **5.0 marks**

Submitted: 2025-11-20 21:03:37 UTC

Chat Link: https://e5-cse-alaaska.vmhost.psu.edu/?chat_id=1dd024cc-3ec3-4f85-ab58-3b45a7d0067e